

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I Santhosh kumar.M(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____ Santhosh_____

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood

System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

1. HDFS-Based Storage Solution for a Media Platform

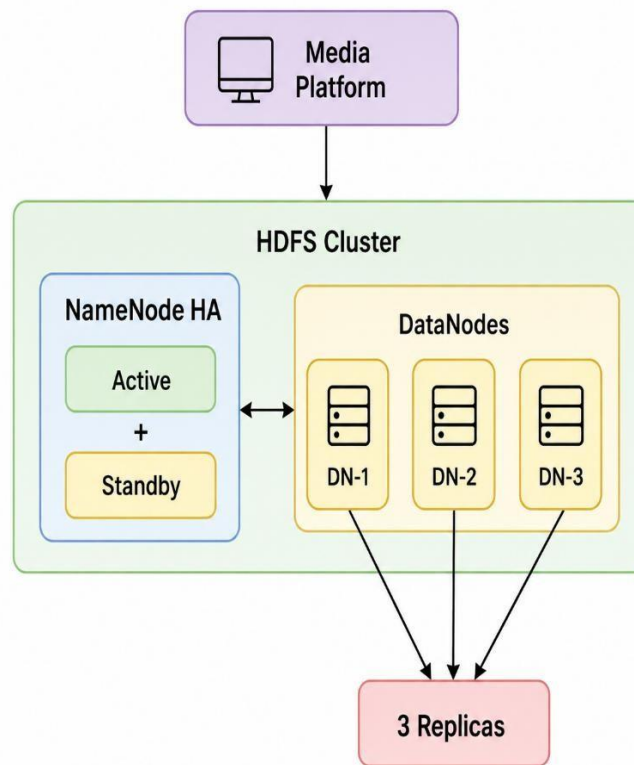
Problem

A media platform stores videos, images, audio, and user-generated content. The system must provide **99.9%+ availability**, use a **maximum replication factor of 3**, and control storage cost.

Proposed Design

- Use **HDFS** as the main storage system.
- Configure **Replication Factor = 3**.
- Store data across multiple **DataNodes**.
- Use **NameNode High Availability** with Active and Standby NameNodes.
- Use **rack-aware data placement** so replicas are stored on different racks.
- Use **compression** for suitable files.
- Move old or rarely accessed media to **low-cost archival storage**.

Architecture



Constraint Satisfaction

Constraint	Solution
Availability > 99.9%	NameNode HA + 3 replicas
Replication ≤ 3	Replication factor = 3
Cost efficiency	Compression + archival storage
Failure protection	Rack-aware replication
Scalability	Add DataNodes when storage grows

Justification

Replication factor 3 provides good fault tolerance without unnecessarily increasing storage consumption. Rack-aware placement protects data even if an entire rack fails. Compression and archival storage reduce the overall cost.

2. MapReduce Workflow for Log Processing

Problem

Large log files must be processed within a **20-minute batch window** while using a limited number of cluster nodes.

Proposed Job Design

Use a two-stage MapReduce workflow:

Log Files

|

↓

Input Splits

|

↓

Mapper Tasks

|

↓

Shuffle & Sort

|

↓

Reducer Tasks

|

↓

Processed Log Report

Mapper

The Mapper reads each log record and extracts useful information such as:

- IP address
- Timestamp
- HTTP status
- URL
- Response time

Example:

Input: /home 200

Output: /home, 1

Reducer

The Reducer combines values from the Mapper.

/home, 1

/home, 1

/home, 1

Output:

/home → 3 requests

Task Distribution

- Divide large logs into multiple **input splits**.
- Assign one Mapper task to each split.
- Run Mapper tasks in parallel across available nodes.
- Use a suitable number of Reducers based on cluster capacity.
- Prefer **data locality** so processing happens near stored data.
- Avoid excessive small files because they create unnecessary tasks.

Meeting the 20-Minute Constraint

Technique	Benefit
Parallel Mapper tasks	Faster processing
Data locality	Reduces network transfer
Appropriate reducers	Prevents reducer bottleneck
Compressed input	Reduces I/O
Limited number of tasks	Avoids task-management overhead

Conclusion

Parallel processing, data locality, and proper Mapper/Reducer distribution allow the log-processing job to complete within the **20-minute batch window** using limited nodes.

3. YARN Scheduler Policy

A YARN cluster runs:

1. ETL jobs
2. Reporting jobs
3. Machine Learning jobs

The scheduler must provide **fairness**, give **priority to ETL**, and recover from **node failures**.

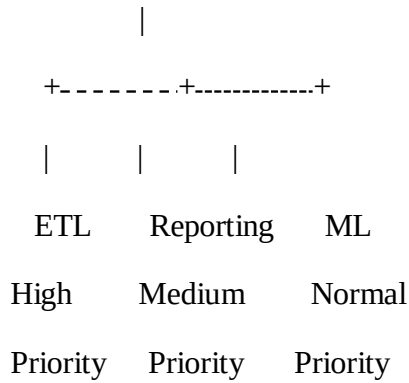
Proposed Scheduler

Use the **Capacity Scheduler** with separate queues.

YARN

|

Capacity Scheduler



Queue Policy

Queue	Priority	Resource Allocation
ETL	High	50%
Reporting	Medium	25%
ML	Normal	25%

Unused resources can be temporarily used by other queues to improve utilization.

Node Failure Recovery

- YARN ResourceManager monitors cluster resources.
- NodeManager sends health information.
- If a node fails, its containers are restarted on healthy nodes.
- Failed tasks are rescheduled.
- Replicated HDFS data remains available.

Constraint Satisfaction

- **Fairness:** Resources are divided among queues.
- **ETL priority:** ETL receives higher priority.
- **Recovery:** Failed containers are automatically rescheduled.
- **Resource utilization:** Idle resources can be shared.

Conclusion

A queue-based YARN policy provides priority for important ETL work while maintaining fair resource sharing and automatic recovery from node failures.

4. Enterprise Data Ingestion Architecture

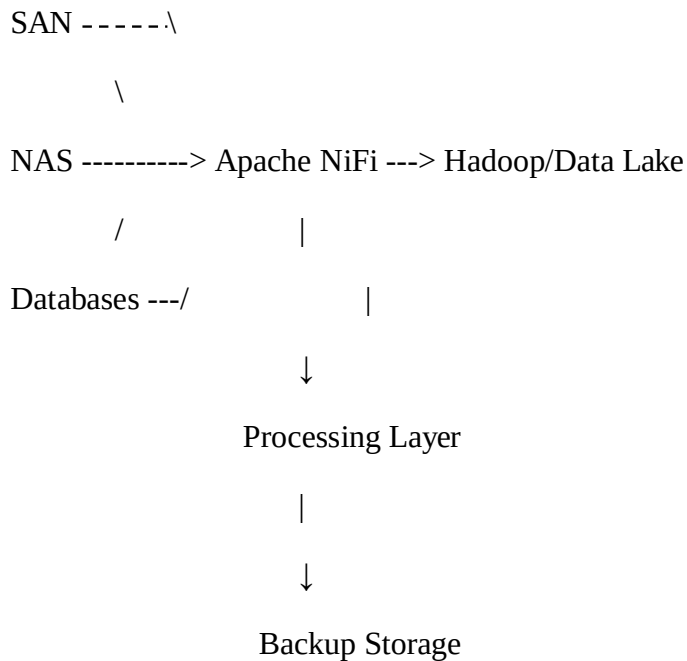
Problem

An enterprise needs to collect data from:

- SAN
- NAS
- Operational databases

The system must provide reliable backup and avoid unnecessary duplicate data.

Proposed Architecture



Components

1. Apache NiFi

- Collects data from different sources.
- Provides data routing and transformation.
- Supports monitoring and flow management.

2. HDFS/Data Lake

- Central storage for large-scale data.
- Provides distributed storage.

- Uses replication for fault tolerance.

3. Database Connectors

- Extract data from operational databases.
- Use incremental extraction where possible.

4. Deduplication

- Use unique IDs, timestamps, or file hashes.
- Check whether data already exists before storing it.

5. Backup

- Maintain replicated HDFS data.
- Use a separate backup/archive location for critical datasets.

Constraint Satisfaction

Constraint	Solution
SAN/NAS ingestion	NiFi connectors and file processors
Database ingestion	JDBC-based ingestion
Backup reliability	HDFS replication + backup storage
Minimal duplication	Hash/ID-based deduplication
Centralized processing	Hadoop data lake

Conclusion

NiFi provides controlled ingestion from multiple enterprise sources, while HDFS provides scalable storage and fault tolerance. Deduplication reduces unnecessary storage consumption.

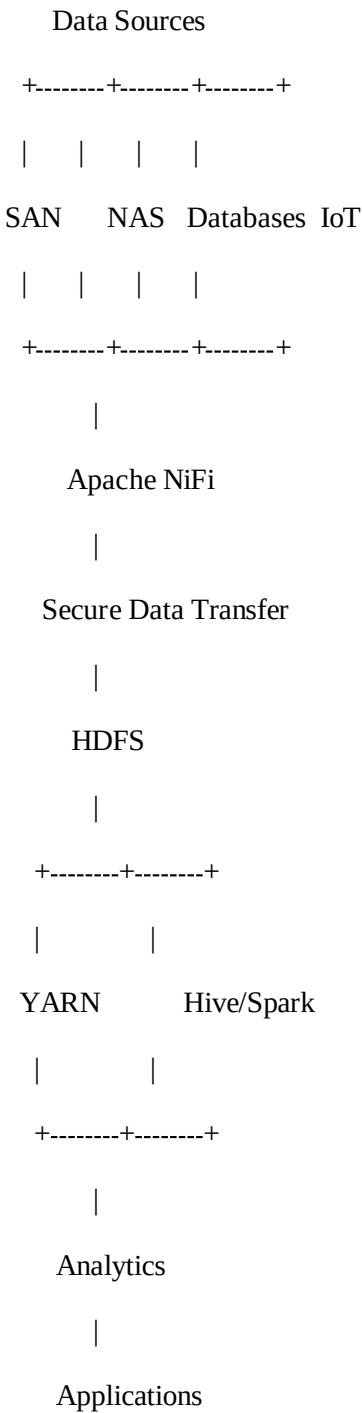
5. End-to-End Hadoop Ecosystem Solution

Requirements

The solution must support:

- Secure data movement
- High availability
- Apache NiFi/ETL integration

Proposed Architecture



Main Components

- 1. Apache NiFi
 - Ingests data from multiple sources.
 - Controls and monitors data flows.
 - Supports ETL operations.
- 2. HDFS

- Stores large datasets.
- Uses replication for fault tolerance.
- Provides scalable storage.

3. YARN

- Manages cluster resources.
- Schedules processing applications.

4. Hive

- Provides SQL-based analysis of Hadoop data.

5. Spark

- Performs fast large-scale data processing and analytics.

Security

- Use **Kerberos authentication**.
- Use **TLS/SSL** for secure data transfer.
- Apply user and group permissions.
- Encrypt sensitive data where required.
- Maintain audit logs.

High Availability

- Use **NameNode Active/Standby**.
- Use multiple DataNodes.
- Use replication factor up to **3**.
- Use YARN ResourceManager HA.
- Automatically reschedule failed processing tasks.

Constraint Satisfaction

Constraint	Solution
Secure data movement	TLS/SSL + authentication
High availability	HA NameNodes + replicated DataNodes
ETL integration	Apache NiFi
Processing	YARN + Spark

Data analysis	Hive
Failure recovery	Replication + task rescheduling

Overall Conclusion

The proposed Hadoop ecosystem provides a **secure, scalable, and highly available big-data platform**. Apache NiFi manages data ingestion and ETL, HDFS provides reliable distributed storage, YARN manages resources, and Hive/Spark support analytics. The design satisfies the constraints while keeping the system efficient and fault tolerant.

